

Architecture - WealthWise

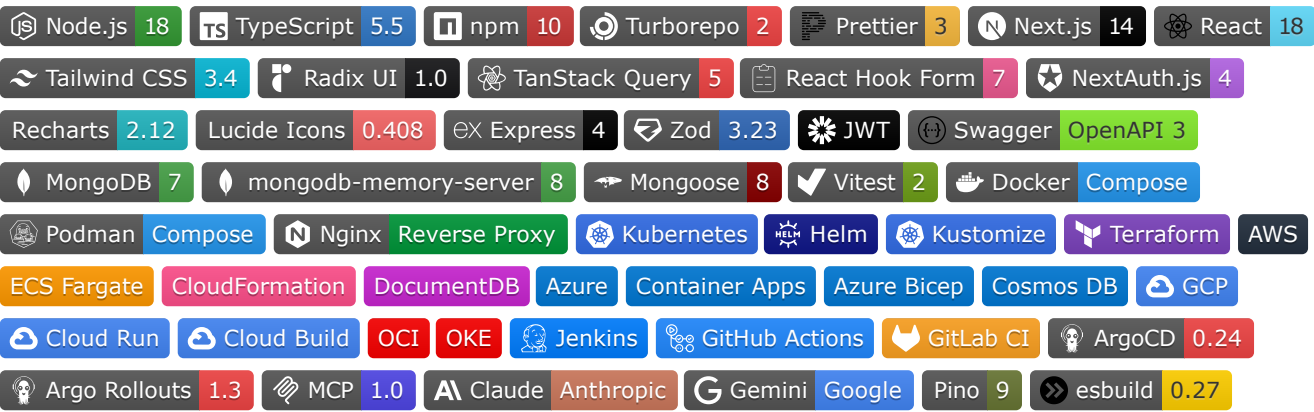
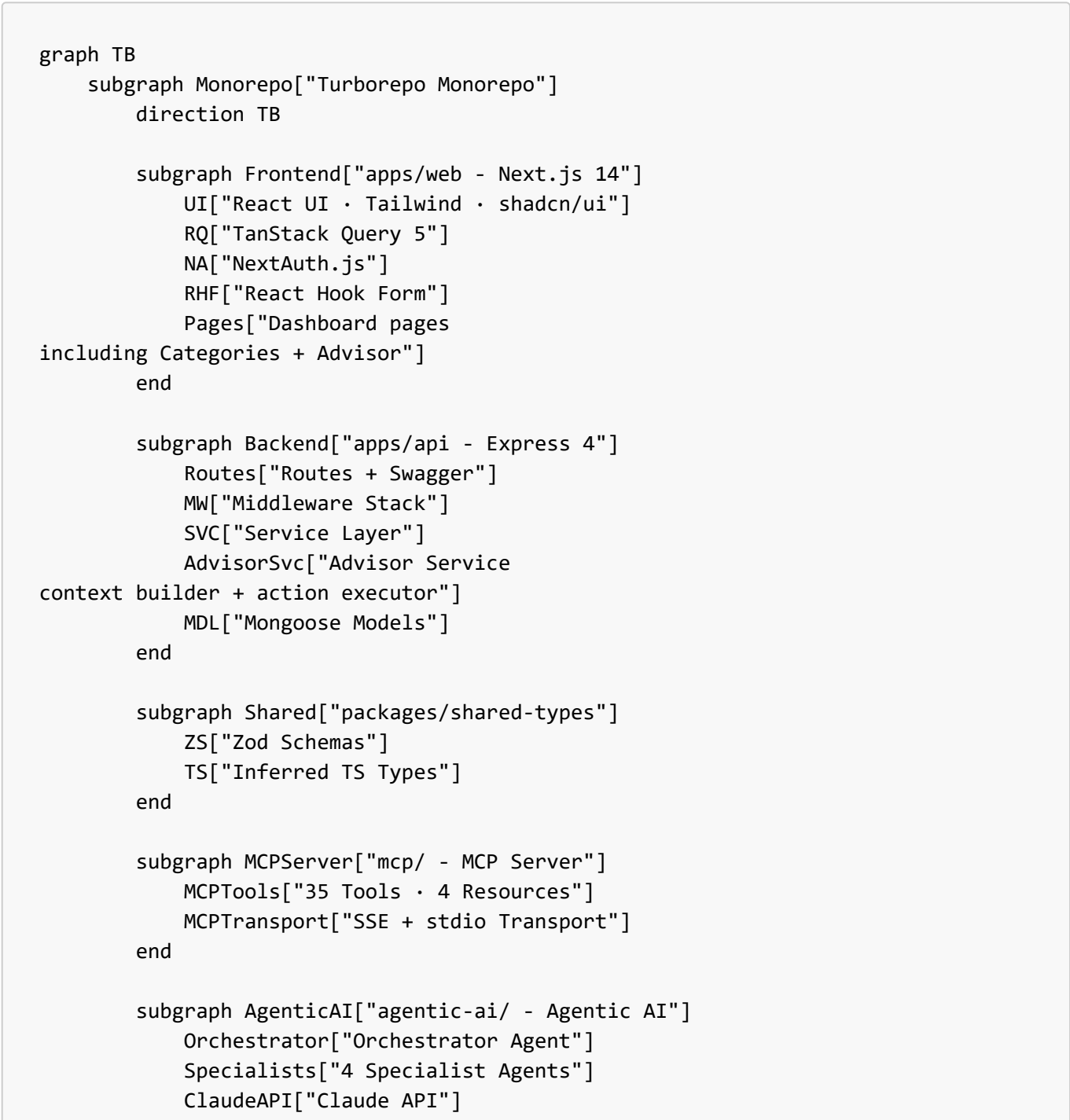


Table of Contents

- [Architecture - WealthWise](#)
 - [Table of Contents](#)
 - [System Overview](#)
 - [Monorepo Build Pipeline](#)
 - [API Architecture](#)
 - [Layered Design](#)
 - [Middleware Pipeline](#)
 - [Route Map](#)
 - [In-App AI Financial Advisor Flow](#)
 - [Error Handling](#)
 - [Pagination](#)
 - [Database Schema](#)
 - [Entity Relationship Diagram](#)
 - [Index Strategy](#)
 - [Web Architecture](#)
 - [Next.js Route Structure](#)
 - [Component Hierarchy](#)
 - [Client Data Flow](#)
 - [State Management](#)
 - [Styling Architecture](#)
 - [Authentication Flow](#)
 - [Full Auth Sequence](#)
 - [Token Lifecycle](#)
 - [Analytics Pipeline](#)
 - [Budget Summary Flow](#)
 - [MCP Server Architecture](#)
 - [Tools \(35 across 7 modules\)](#)
 - [Resources \(4\)](#)
 - [Auth Flow](#)
 - [Agentic AI Architecture](#)

- Agent Pipeline
- Agents
- Chat Request Lifecycle
- Deployment Architecture
 - Production (Docker/Podman Compose)
 - Nginx Routing
 - Development vs Production
- Shared Types Contract
 - Schema Coverage
- Testing Strategy
 - Test Execution Flow

System Overview



```

    end
  end

  DB[(MongoDB 7)]
  Gemini["Gemini API"]

  UI --> RQ
  Pages --> RQ
  RQ -- "HTTP / REST" --> Routes
  NA -- "Bearer Token" --> MW
  RHF -. "validates forms" .-> ZS
  Routes --> MW --> SVC --> MDL --> DB
  MW --> AdvisorSvc
  AdvisorSvc --> MDL
  AdvisorSvc --> Gemini
  MW -. "validates input" .-> ZS
  ZS --> TS
  Orchestrator --> Specialists --> ClaudeAPI
  ClaudeAPI -- "tool_use" --> MCPTransport
  MCPTransport --> MCPTools --> DB

  style Frontend fill:#0f172a,stroke:#6366f1,color:#e2e8f0
  style Backend fill:#0f172a,stroke:#10b981,color:#e2e8f0
  style Shared fill:#0f172a,stroke:#f59e0b,color:#e2e8f0
  style MCPServer fill:#0f172a,stroke:#4f46e5,color:#e2e8f0
  style AgenticAI fill:#0f172a,stroke:#cc785c,color:#e2e8f0
  style DB fill:#0f172a,stroke:#47a248,color:#e2e8f0
  style Gemini fill:#0f172a,stroke:#4285f4,color:#e2e8f0

```

The application follows a **client-server architecture** inside a **Turborepo monorepo**. The frontend and backend are fully decoupled, communicating over a REST API, with a shared package providing Zod schemas that enforce type safety at both boundaries.

There are now two distinct AI surfaces in the system:

- The in-app `/advisor` experience is served by `apps/api`, which builds a live finance context from MongoDB and calls Gemini for grounded chat plus confirmation-based action planning.
- The separate `agentic-ai/` service remains Claude-powered and MCP-backed for specialist agent workflows.

[!NOTE] The frontend is deployed on Vercel at: <https://wealthwisefinancial.vercel.app/>. You can register a new account or use the following demo credentials to explore the app:

```

Email: demo@wealthwise.app
Password: Demo1234!

```

Or, create your own account to test the registration flow and see how the app works with an empty dataset.

[!TIP] The backend is also fully deployed live, accessible at: <https://wealthwise-backend-api.vercel.app/>. You can explore the API documentation at <https://wealthwise-backend-api.vercel.app/api/docs> and use the demo credentials above to authenticate and test the endpoints.

Monorepo Build Pipeline

```
graph LR
  ST["packages/shared-types"] --> API["apps/api"]
  ST --> WEB["apps/web"]
  MCP_PKG["mcp/"] --> DB_DIRECT["MongoDB (direct)"]
  AI_PKG["agentic-ai/"] --> MCP_PKG

  subgraph Turbo["Turborepo Pipeline"]
    direction LR
    Build["build"]
    dependsOn: ^build"] --> Lint["lint"]
    Build --> Test["test"]
    Dev["dev"]
    persistent: true"]
  end

  style ST fill:#f59e0b,stroke:#000,color:#000
  style API fill:#10b981,stroke:#000,color:#000
  style WEB fill:#6366f1,stroke:#000,color:#fff
  style MCP_PKG fill:#4f46e5,stroke:#000,color:#fff
  style AI_PKG fill:#cc785c,stroke:#000,color:#fff
```

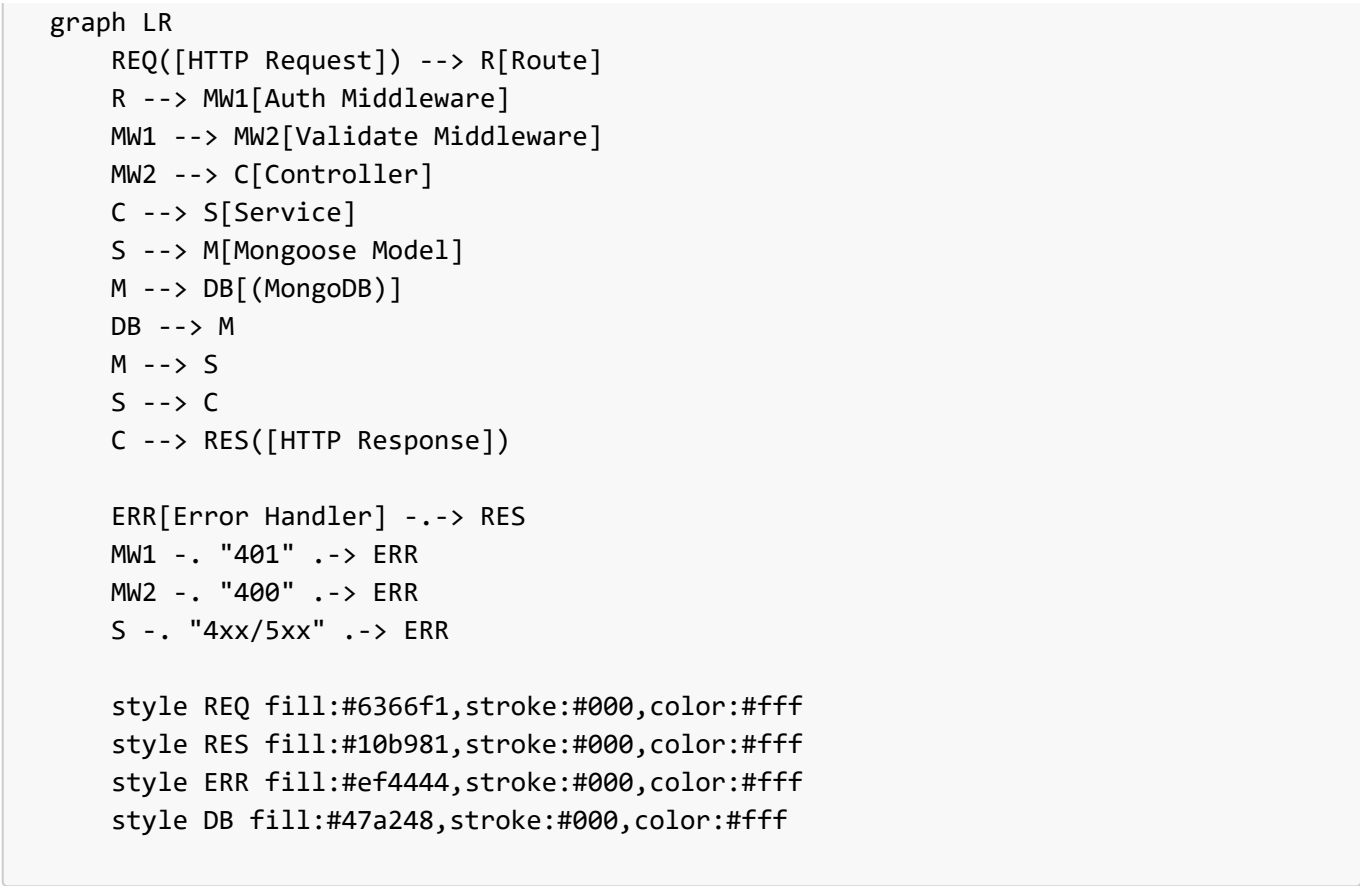
Task	dependsOn	Outputs	Cached
build	^build (shared-types first)	.next/**, dist/**	Yes
dev	-	-	No
lint	-	-	Yes
test	-	-	Yes
clean	-	-	No

apps/web and apps/api both depend on packages/shared-types. Turbo builds shared-types first, then builds both apps in parallel. mcp/ and agentic-ai/ are independent packages that do not depend on shared-types; agentic-ai/ connects to the MCP server at runtime.

API Architecture

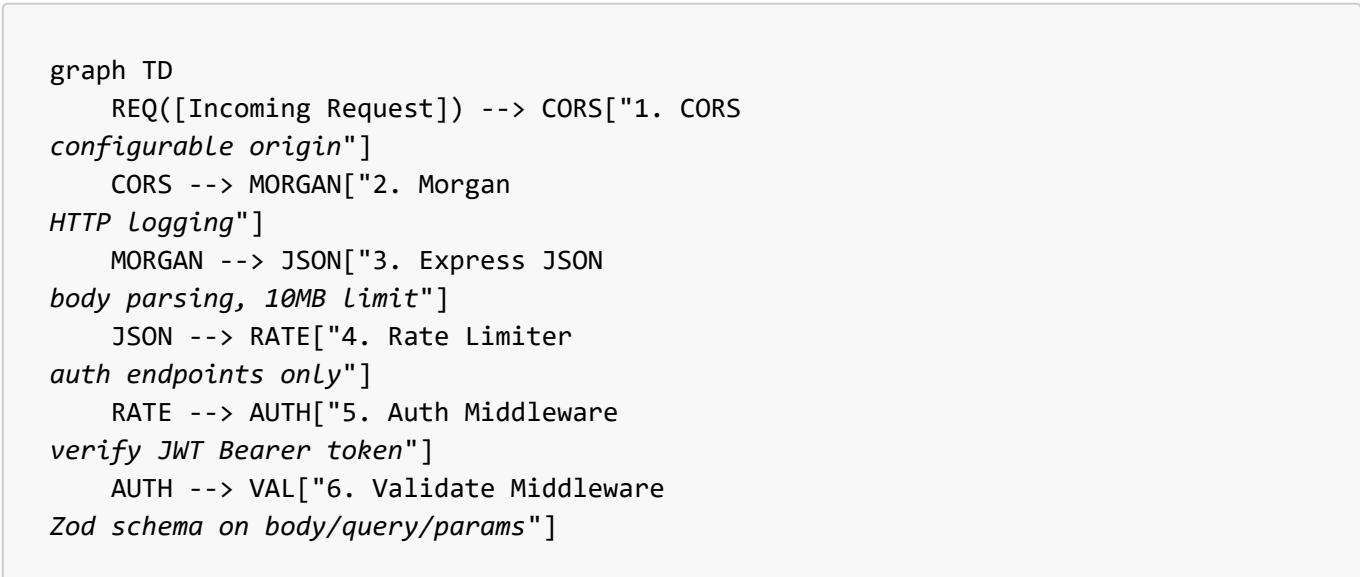
Layered Design





Layer	Responsibility	Location
Routes	URL mapping, Swagger JSDoc annotations	src/routes/
Middleware	Auth, validation, CORS, rate limiting, errors	src/middleware/
Controllers	Parse request, call service, format response	src/controllers/
Services	Business logic, data access, aggregations	src/services/
Models	Mongoose schemas, indexes, virtuals	src/models/
Utils	ApiError class, async handler, pagination	src/utils/

Middleware Pipeline



```
VAL --> HANDLER["7. Route Handler"]
HANDLER --> ERRH["Error Handler
global catch-all"]
ERRH --> RES([Response])

AUTH -. "unauthorized" .-> ERRH
VAL -. "validation error" .-> ERRH

style REQ fill:#6366f1,stroke:#000,color:#fff
style RES fill:#10b981,stroke:#000,color:#fff
style ERRH fill:#ef4444,stroke:#000,color:#fff
```

Route Map

All routes are prefixed with */api/v1*.

Resource	Routes	Controller	Service
Auth	<i>auth.routes.ts</i>	<i>auth.controller.ts</i>	<i>auth.service.ts</i>
Accounts	<i>account.routes.ts</i>	<i>account.controller.ts</i>	<i>account.service.ts</i>
Transactions	<i>transaction.routes.ts</i>	<i>transaction.controller.ts</i>	<i>transaction.service.ts</i>
Categories	<i>category.routes.ts</i>	<i>category.controller.ts</i>	<i>category.service.ts</i>
Budgets	<i>budget.routes.ts</i>	<i>budget.controller.ts</i>	<i>budget.service.ts</i>
Goals	<i>goal.routes.ts</i>	<i>goal.controller.ts</i>	<i>goal.service.ts</i>
Recurring	<i>recurring.routes.ts</i>	<i>recurring.controller.ts</i>	<i>recurring.service.ts</i>
Analytics	<i>analytics.routes.ts</i>	<i>analytics.controller.ts</i>	<i>analytics.service.ts</i>
Advisor	<i>advisor.routes.ts</i>	<i>advisor.controller.ts</i>	<i>advisor.service.ts</i>

In-App AI Financial Advisor Flow

```
sequenceDiagram
    participant U as User
    participant WEB as Next.js /advisor
    participant API as Express API
    participant AUTH as JWT Auth
    participant ADV as advisor.service
    participant DB as MongoDB
    participant GEM as Gemini API

    U->>WEB: Ask for analysis or a create action
    WEB->>API: POST /api/v1/advisor/chat
    API->>AUTH: Verify bearer token
    AUTH-->>API: userId
    API->>ADV: chat(userId, message, history)
    ADV->>DB: Load accounts, transactions, categories, budgets, goals, recurring
```

```
rules
  DB-->>ADV: Live finance data
  ADV-->>GEM: Structured prompt + finance context + frontend workflow guide
  GEM-->>ADV: reply-only JSON
  ADV-->>WEB: grounded reply with analysis or exact in-app steps
```

The advisor never executes model-proposed actions automatically. Gemini can only propose structured create actions, and the frontend must explicitly confirm them before the API calls the underlying account, category, budget, goal, recurring, or transaction service.

Error Handling

All errors flow through a single global error handler. The custom `ApiError` class provides factory methods:

```
graph LR
  subgraph Sources["Error Sources"]
    ZE["Zod Validation Error"]
    ME["Mongoose Cast Error"]
    AE["ApiError"]
    UE["Unhandled Error"]
  end

  subgraph Handler["Global Error Handler"]
    NORM["Normalize to standard shape"]
  end

  ZE --> NORM
  ME --> NORM
  AE --> NORM
  UE --> NORM

  NORM --> R400["400 Bad Request"]
  NORM --> R401["401 Unauthorized"]
  NORM --> R404["404 Not Found"]
  NORM --> R409["409 Conflict"]
  NORM --> R500["500 Internal"]

  style R400 fill:#f59e0b,stroke:#000,color:#000
  style R401 fill:#ef4444,stroke:#000,color:#fff
  style R404 fill:#6366f1,stroke:#000,color:#fff
  style R409 fill:#f97316,stroke:#000,color:#000
  style R500 fill:#dc2626,stroke:#000,color:#fff
```

Response shape: { success: false, error: { code, message, details? } }

Pagination

Every list endpoint supports:

Param	Default	Max
-------	---------	-----

Param	Default	Max
page	1	-
limit	20	100

Response includes { data, pagination: { page, limit, total, totalPages } }.

Database Schema

Entity Relationship Diagram

```
erDiagram
    USER ||--o{ ACCOUNT : owns
    USER ||--o{ TRANSACTION : owns
    USER ||--o{ CATEGORY : creates
    USER ||--o{ BUDGET : sets
    USER ||--o{ GOAL : tracks
    USER ||--o{ RECURRING_RULE : defines

    ACCOUNT ||--o{ TRANSACTION : contains
    CATEGORY ||--o{ TRANSACTION : categorizes
    CATEGORY ||--o{ BUDGET : "spending limit for"
    ACCOUNT ||--o{ RECURRING_RULE : "charges to"
    CATEGORY ||--o{ RECURRING_RULE : categorizes

    USER {
        ObjectId _id PK
        string email UK
        string name
        string passwordHash
        string avatarUrl
        string currency
    }

    ACCOUNT {
        ObjectId _id PK
        ObjectId userId FK
        string name
        enum type "checking|savings|credit_card|cash|investment"
        number balance
        string color
        boolean isArchived
    }

    TRANSACTION {
        ObjectId _id PK
        ObjectId userId FK
        ObjectId accountId FK
        ObjectId categoryId FK
        enum type "income|expense|transfer"
```



```
    number amount
    string description
    date date
    string[] tags
}

CATEGORY {
    ObjectId _id PK
    ObjectId userId FK "null = system default"
    string name
    string icon
    string color
    enum type "income|expense"
    boolean isDefault
}

BUDGET {
    ObjectId _id PK
    ObjectId userId FK
    ObjectId categoryId FK
    number amount
    enum period "monthly|weekly"
    number alertThreshold
    boolean isActive
}

GOAL {
    ObjectId _id PK
    ObjectId userId FK
    string name
    number targetAmount
    number currentAmount
    date deadline
    boolean isCompleted
}

RECURRING_RULE {
    ObjectId _id PK
    ObjectId userId FK
    ObjectId accountId FK
    ObjectId categoryId FK
    enum frequency "daily|weekly|biweekly|monthly|yearly"
    number amount
    date nextDueDate
    boolean isActive
}
```

All user-owned entities store a `userId` foreign key. Ownership is enforced at the service layer - every query filters by `userId` from the JWT.

Index Strategy

```

graph TD
  subgraph Transaction_Indexes
    TI1["userId + date ↓  
default sort"]
    TI2["userId + categoryId + date ↓  
category queries"]
    TI3["userId + accountId + date ↓  
account queries"]
    TI4["description: text  
full-text search"]
  end

  subgraph Other_Indexes
    AI["Account: userId + isArchived"]
    CI["Category: userId + type"]
    BI["Budget: userId + categoryId"]
    RI["RecurringRule: userId + isActive + nextDueDate"]
    UI["User: email (unique)"]
  end

```

Web Architecture

Next.js Route Structure

```

graph TD
  ROOT["/ - Root Layout  
Providers: Query, Auth, Theme, UI Preferences"]
  LANDING["/ - Landing Page  
public, marketing"]
  NOTFOUND["404 - Not Found"]
  AUTH_GROUP["(auth) - Auth Layout  
minimal, no sidebar"]
  DASH_GROUP["(dashboard) - Dashboard Layout  
sidebar + topnav"]
  LEGAL_GROUP["(legal) - Legal Layout  
nav + footer"]
  NEXTAUTH["/api/auth/[...nextauth]  
NextAuth handler"]
  LOGIN["/login"]
  REGISTER["/register"]
  DASHBOARD["/dashboard"]
  TRANSACTIONS["/transactions"]
  CATEGORIES["/categories"]
  BUDGETS["/budgets"]
  GOALS["/goals"]
  ACCOUNTS["/accounts"]
  RECURRING["/recurring"]

  ROOT --> LANDING
  ROOT --> NOTFOUND
  ROOT --> AUTH_GROUP
  ROOT --> DASH_GROUP
  ROOT --> LEGAL_GROUP
  ROOT --> NEXTAUTH
  AUTH_GROUP --> LOGIN
  AUTH_GROUP --> REGISTER
  DASH_GROUP --> DASHBOARD
  DASH_GROUP --> TRANSACTIONS
  DASH_GROUP --> CATEGORIES
  DASH_GROUP --> BUDGETS
  DASH_GROUP --> GOALS
  DASH_GROUP --> ACCOUNTS
  DASH_GROUP --> RECURRING

```

```

DASH_GROUP --> ANALYTICS["/analytics"]
DASH_GROUP --> ADVISOR["/advisor"]
DASH_GROUP --> SETTINGS["/settings"]

LEGAL_GROUP --> TERMS["/terms"]
LEGAL_GROUP --> PRIVACY["/privacy"]

style ROOT fill:#6366f1,stroke:#000,color:#fff
style AUTH_GROUP fill:#f59e0b,stroke:#000,color:#000
style DASH_GROUP fill:#10b981,stroke:#000,color:#fff
style LEGAL_GROUP fill:#8b5cf6,stroke:#000,color:#fff

```

Route groups (([auth](#)), ([dashboard](#)), ([legal](#))) share layouts without affecting URL structure.

Component Hierarchy

```

graph TD
    subgraph UI["components/ui - shadcn/ui Primitives"]
        BTN[Button] & CARD[Card] & DLG[Dialog] & INP[Input]
        SEL[Select] & TAB[Tabs] & BADGE[Badge] & PROG[Progress]
    end

    subgraph Layout["components/layout"]
        SIDE[Sidebar] & TOP[Topnav] & MOB[MobileNav] & SEARCH[SearchCommand]
    end

    subgraph Features["Feature Components"]
        subgraph Dash["dashboard/"]
            NW[NetWorthCard] & MS[MonthlySnapshot] & RT[RecentTransactions]
            BH[BudgetHealth] & SD[SpendingDonut] & UB[UpcomingBills] &
            GP[GoalProgress]
        end
        subgraph Cats["categories/"]
            CC[CategoryCard] & CS[CategorySection] & CFD[CategoryFormDialog] &
            CDD[CategoryDeleteDialog]
        end
        subgraph Adv["advisor/"]
            AM[AdvisorMessage] & ASC[AdvisorSuggestionCard]
        end
        subgraph SettingsComp["settings/"]
            AT[AppearanceTab]
        end
        subgraph Trans["transactions/"]
            TT[TransactionTable] & TF[TransactionForm] & CSV[CsvImportWizard] &
            FS[FilterSidebar]
        end
        subgraph Charts["analytics/"]
            SC[SpendingChart] & TC[TrendChart] & NWC[NetWorthChart] &
            SRC[SavingsRateChart]
            CFC[CashFlowChart] & CSC[CumulativeSavingsChart] & DWC[DayOfWeekChart]
            & CBC[CategoryBreakdownChart]
        end
    end

```

```
        end
    end

    subgraph SharedComp["components/shared"]
        AP[AccountPicker] & CP[CategoryPicker] & DRP[DateRangePicker] &
ES[EmptyState]
    end

    Features --> SharedComp --> UI
    Layout --> UI
```

Client Data Flow

```
sequenceDiagram
    participant C as React Component
    participant H as TanStack Query Hook
    participant AC as API Client
    participant S as NextAuth Session
    participant API as Express API
    participant DB as MongoDB

    C->>H: useTransactions()
    H->>S: getSession()
    S-->>H: { accessToken }
    H->>AC: apiClient.get("/transactions")
    AC->>API: GET /api/v1/transactions
    Authorization: Bearer {token}
    API->>DB: Transaction.find({ userId })
    DB-->>API: documents[]
    API-->>AC: { success: true, data, pagination }
    AC-->>H: response data
    H-->>C: { data, isLoading, error }

    Note over C,H: Mutations trigger cache
    invalidation + refetch

    C->>H: useCreateTransaction()
    H->>AC: apiClient.post("/transactions", body)
    AC->>API: POST /api/v1/transactions
    API->>DB: Transaction.create(...)
    DB-->>API: new document
    API-->>AC: { success: true, data }
    AC-->>H: response
    H->>H: invalidateQueries(["transactions"])
    H-->>C: Toast: "Transaction created"
```

State Management

Concern	Solution
---------	----------

Concern	Solution
Server state	TanStack Query 5 (cache, refetch, optimistic updates)
Form state	React Hook Form + Zod resolver
Auth session	NextAuth.js (JWT strategy)
Theme	next-themes (class-based, <i>localStorage</i>)
UI preferences	Local provider for appearance, reduced motion, contrast
URL state	Next.js <i>searchParams</i> for filters

No global client-side store (Redux, Zustand, etc.) is used. TanStack Query serves as the single source of truth for all server-derived state, while lightweight client preferences such as appearance and motion settings are persisted locally.

Styling Architecture

```
graph TD
    TW["Tailwind CSS 3.4"] --> HSL["HSL CSS Variables  
globals.css :root / .dark"]
    TW --> SHADCN["shadcn/ui Components  
Radix UI primitives"]
    TW --> ANIM["Custom Animations  
accordion, fade, slide, shimmer"]

    NT["next-themes"] --> DARK["Dark Mode  
.dark class on html"]
    DARK --> HSL

    HSL --> COLORS["Theme Colors  
--background, --foreground  
--card, --primary, --border"]
    HSL --> SIDEBAR_COLORS["Sidebar Colors  
--sidebar-background  
--sidebar-border"]

    style TW fill:#06b6d4,stroke:#000,color:#000
    style NT fill:#8b5cf6,stroke:#000,color:#fff
```

Authentication Flow

Full Auth Sequence

```
sequenceDiagram
    participant U as User
    participant WEB as Next.js Frontend
    participant NA as NextAuth.js
```

```

participant API as Express API
participant DB as MongoDB

rect rgb(30, 41, 59)
  Note over U,DB: Registration
  U->>WEB: Fill register form
  WEB->>API: POST /api/v1/auth/register
{email, name, password}
  API->>API: Validate with Zod
  API->>API: Hash password (bcrypt 12 rounds)
  API->>DB: Create User document
  DB-->>API: User created
  API->>API: Sign JWT (15min) + Refresh (7d)
  API-->>WEB: { user, accessToken, refreshToken }
  WEB->>WEB: Redirect to /login
end

rect rgb(30, 41, 59)
  Note over U,DB: Login
  U->>WEB: Submit credentials
  WEB->>NA: signIn("credentials", { email, password })
  NA->>API: POST /api/v1/auth/login
  API->>DB: Find user by email
  DB-->>API: User document
  API->>API: Compare bcrypt hash
  API->>API: Generate token pair
  API-->>NA: { user, accessToken, refreshToken }
  NA->>NA: jwt callback: store tokens
  NA->>NA: session callback: expose to client
  NA-->>WEB: Session with accessToken
  WEB->>WEB: Redirect to /dashboard
end

rect rgb(30, 41, 59)
  Note over U,DB: Authenticated Request
  U->>WEB: Navigate to /transactions
  WEB->>NA: getSession()
  NA-->>WEB: { accessToken, user }
  WEB->>API: GET /api/v1/transactions
Authorization: Bearer {token}
  API->>API: Verify JWT signature + expiry
  API->>API: Extract userId from payload
  API->>DB: Transaction.find({ userId })
  DB-->>API: Results
  API-->>WEB: { success: true, data }
end

```

Token Lifecycle

```

graph LR
  subgraph Tokens

```

```

    AT["Access Token
15 minutes"]
    RT["Refresh Token
7 days"]
    NS["NextAuth Session
7 days"]
    end

    subgraph Storage
        JWT["httpOnly Cookie
(NextAuth JWT)"]
    end

    AT --> JWT
    RT --> JWT
    NS --> JWT

    JWT -- "auto-injected" --> API["API Client
Authorization: Bearer"]

    style AT fill:#f59e0b,stroke:#000,color:#000
    style RT fill:#10b981,stroke:#000,color:#fff
    style NS fill:#6366f1,stroke:#000,color:#fff

```

Analytics Pipeline

```

graph TD
    subgraph Endpoints["Analytics Endpoints"]
        E1["GET /spending-by-category"]
        E2["GET /income-vs-expense"]
        E3["GET /monthly-summary"]
        E4["GET /trends"]
        E5["GET /net-worth"]
        E6["GET /spending-by-day-of-week"]
        E7["GET /category-monthly-breakdown"]
    end

    subgraph Pipeline["MongoDB Aggregation Pipeline"]
        MATCH["$match
userId + date range"]
        GROUP["$group
by category / month / type"]
        LOOKUP["$lookup
join category names"]
        PROJECT["$project
compute totals, ratios"]
        SORT["$sort
chronological / by amount"]
    end

```

```

E1 & E2 & E3 & E4 & E5 & E6 & E7 --> MATCH
MATCH --> GROUP --> LOOKUP --> PROJECT --> SORT

subgraph Results["Frontend Visualization"]
  R1["Spending Donut Chart"]
  R2["Income vs Expense Bars"]
  R3["Monthly Summary Cards"]
  R4["Savings Rate Line Chart"]
  R5["Net Worth Area Chart"]
  R6["Cash Flow Area Chart"]
  R7["Cumulative Savings Chart"]
  R8["Category Breakdown Stacked Bars"]
  R9["Day of Week Bar Chart"]
end

SORT --> R1 & R2 & R3 & R4 & R5 & R6 & R7 & R8 & R9

style Pipeline fill:#0f172a,stroke:#47a248,color:#e2e8f0

```

Budget Summary Flow

```

graph LR
  BQ["GET /budgets/summary"] --> M1["$match  
userId + period dates"]
  M1 --> G1["$group by categoryId  
sum(amount) as spent"]
  G1 --> J1["Join with Budget.amount"]
  J1 --> CALC1["Compute ratio  
spent / budget"]
  CALC1 --> S1["under_budget  
ratio < threshold"]
  CALC1 --> S2["warning  
ratio >= threshold"]
  CALC1 --> S3["over_budget  
ratio > 1.0"]

  style S1 fill:#10b981,stroke:#000,color:#fff
  style S2 fill:#f59e0b,stroke:#000,color:#000
  style S3 fill:#ef4444,stroke:#000,color:#fff

```

MCP Server Architecture

The MCP server implements the [Model Context Protocol](#) to expose WealthWise's financial data to AI agents.

```

graph LR
  CLIENT["MCP Client  
(Agentic AI / external)"] --> TRANSPORT["Transport Layer  
SSE :5100 · stdio"]

```



```
TRANSPORT --> SERVER["McpServer
@modelcontextprotocol/sdk"]
SERVER --> TOOLS["Tools (35)"]
SERVER --> RES["Resources (4)"]
TOOLS --> MODELS["Mongoose Models"]
RES --> MODELS
MODELS --> DB[(MongoDB)]

AUTH["JWT Token Resolver"] -. "userId" .-> TOOLS
AUTH -. "userId" .-> RES

style CLIENT fill:#6366f1,stroke:#000,color:#fff
style TRANSPORT fill:#4f46e5,stroke:#000,color:#fff
style SERVER fill:#4f46e5,stroke:#000,color:#fff
style DB fill:#47a248,stroke:#000,color:#fff
```

Tools (35 across 7 modules)

Module	Tools	Examples
Accounts	5	list_accounts, get_account, create_account, update_account, delete_account
Transactions	5	list_transactions, get_transaction, create_transaction, update_transaction, delete_transaction
Categories	5	list_categories, get_category, create_category, update_category, delete_category
Budgets	5	list_budgets, get_budget, create_budget, update_budget, delete_budget
Goals	5	list_goals, get_goal, create_goal, update_goal, add_funds_to_goal
Recurring	5	list_recurring_rules, get_recurring_rule, create_recurring_rule, update_recurring_rule, delete_recurring_rule
Analytics	5	spending_by_category, income_vs_expense, monthly_summary, net_worth, spending_trends

Resources (4)

Resource URI	Description
wealthwise://schema/accounts	Account schema metadata
wealthwise://schema/transactions	Transaction schema metadata
wealthwise://summary/accounts	Account summary with balances
wealthwise://summary/budgets	Budget summary with spending progress

Auth Flow

```
sequenceDiagram
    participant Client as MCP Client
    participant Transport as SSE Transport
    participant Auth as Token Resolver
    participant Server as McpServer
    participant DB as MongoDB

    Client->>Transport: Connect (Bearer token in header)
    Transport->>Auth: Resolve JWT
    Auth->>Auth: Verify signature + expiry
    Auth-->>Transport: userId
    Client->>Transport: tools/call (list_accounts)
    Transport->>Server: Execute tool (userId context)
    Server->>DB: Account.find({ userId })
    DB-->>Server: Documents
    Server-->>Transport: Tool result (JSON)
    Transport-->>Client: SSE event
```

Agentic AI Architecture

The Agentic AI service provides a conversational financial advisor using Claude Sonnet 4 with tool-use capabilities. It is separate from the in-app [/advisor](#) flow in [apps/api](#), which uses Gemini and direct database-backed finance context assembly instead of MCP tool calls.

Agent Pipeline

```
graph LR
    USER["User Message"] --> REST["REST API :5200"]
    REST --> AUTH["Auth Middleware  
JWT verification"]
    AUTH --> ORCH["Orchestrator  
Intent Classifier"]
    ORCH --> FA["Financial Advisor"]
    ORCH --> AD["Anomaly Detector"]
    ORCH --> BO["Budget Optimizer"]
    ORCH --> FC["Forecaster"]
    FA & AD & BO & FC --> CLAUDE["Claude API  
tool_use loop"]
    CLAUDE --> MCPC["MCP Client"]
    MCPC --> MCP["MCP Server :5100"]
    MCP --> DB["MongoDB"]
    CLAUDE --> RESP["Response"]
    RESP --> USER

    style USER fill:#6366f1,stroke:#000,color:#fff
    style ORCH fill:#cc785c,stroke:#000,color:#fff
    style CLAUDE fill:#cc785c,stroke:#000,color:#fff
```

```
style MCP fill:#4f46e5,stroke:#000,color:#fff
style DB fill:#47a248,stroke:#000,color:#fff
```

Agents

Agent	Role	Key MCP Tools Used
Orchestrator	Classifies intent, routes to specialist	None (classification only)
Financial Advisor	General financial advice, account overview	list_accounts, list_transactions, net_worth
Anomaly Detector	Identifies unusual spending patterns	list_transactions, spending_by_category, spending_trends
Budget Optimizer	Recommends budget adjustments	list_budgets, spending_by_category, income_vs_expense
Forecaster	Projects future spending and savings	monthly_summary, spending_trends, net_worth

Chat Request Lifecycle

```
sequenceDiagram
    participant U as User
    participant API as Agentic AI :5200
    participant Auth as Auth Middleware
    participant Orch as Orchestrator
    participant Agent as Specialist Agent
    participant Claude as Claude API
    participant MCP as MCP Server :5100
    participant DB as MongoDB

    U->>API: POST /agent/chat { message }
    API->>Auth: Verify JWT
    Auth-->>API: userId

    API->>Orch: Classify intent
    Orch->>Claude: System prompt + user message
    Claude-->>Orch: Classification (e.g., "budget_optimization")
    Orch->>Agent: Delegate to Budget Optimizer

    loop Tool-use iterations
        Agent->>Claude: System prompt + conversation + tools
        Claude-->>Agent: tool_use: spending_by_category
        Agent->>MCP: Call tool (userId context)
        MCP->>DB: Aggregation query
        DB-->>MCP: Results
        MCP-->>Agent: Tool result
        Agent->>Claude: Tool result appended
    end
```

```
end

Claude-->>Agent: Final text response
Agent-->>API: Response + conversation state
API-->>U: { message, agent, toolsUsed }
```

Deployment Architecture

Production (Docker/Podman Compose)

```
graph TD
    CLIENT([Browser]) -- ":80 / :443" --> NGINX[NGinx Reverse Proxy  
TLS · gzip · headers]
    NGINX -- "/api/" --> API[API Container  
Express :4000  
production build]
    NGINX -- "/mcp/" --> MCP[MCP Container  
MCP Server :5100  
production build]
    NGINX -- "/agent/" --> AI[Agentic AI Container  
:5200  
production build]
    NGINX -- "/" --> WEB[Web Container  
Next.js :3000  
standalone build]
    API --> DB[(MongoDB :27017  
persistent volume)]
    MCP --> DB
    AI -- "MCP protocol" --> MCP

    subgraph NETWORK["wealthwise-network (bridge)"]
        NGINX
        API
        WEB
        MCP
        AI
        DB
    end

    style CLIENT fill:#6366f1,stroke:#000,color:#fff
    style NGINX fill:#009639,stroke:#000,color:#fff
    style API fill:#10b981,stroke:#000,color:#fff
    style WEB fill:#0f172a,stroke:#6366f1,color:#e2e8f0
    style MCP fill:#4f46e5,stroke:#000,color:#fff
    style AI fill:#cc785c,stroke:#000,color:#fff
    style DB fill:#47a248,stroke:#000,color:#fff
    style NETWORK fill:#1e293b,stroke:#475569,color:#94a3b8
```

Nginx Routing

- `/api/` → upstream `api:4000`
- `/mcp/` → upstream `mcp:5100`
- `/agent/` → upstream `ai:5200`
- `/` → upstream `web:3000`
- Gzip enabled for text, JSON, JS, CSS, SVG
- Client max body size: 10MB (for CSV imports)
- Proxy headers: `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`

Development vs Production

```
graph LR
    subgraph Dev["docker-compose.yml / podman-compose.yml"]
        D_MONGO["MongoDB :27017"]
        D_API["API :4000"]
        D_WEB["Web :3000"]
        D_MCP["MCP :5100"]
        D_AI["Agentic AI :5200"]
    end
    Dev -- "build & deploy" --> Prod

    subgraph Prod["docker-compose.prod.yml / podman-compose.prod.yml"]
        P_NGINX["Nginx :80/:443"]
        P_MONGO["MongoDB :27017"]
        P_API["API :4000"]
        P_WEB["Web :3000"]
        P_MCP["MCP :5100"]
        P_AI["Agentic AI :5200"]
    end

    style Dev fill:#1e293b,stroke:#f59e0b,color:#e2e8f0
    style Prod fill:#1e293b,stroke:#10b981,color:#e2e8f0
```

Shared Types Contract

```

graph TD
    subgraph PKG["packages/shared-types"]
        ZOD["Zod Schemas  
runtime validation rules"]
        INFER["z.infer<typeof schema>"]
        ZOD --> INFER
        INFER --> TYPES["TypeScript Types  
compile-time safety"]
    end

    subgraph API_USE["apps/api"]
        VAL["validate() middleware  
body / query / params"]
        RESP["Response types"]
    end

    subgraph WEB_USE["apps/web"]
        FORM["zodResolver()  
React Hook Form"]
        HOOKS["Query/Mutation types"]
    end

    ZOD --> VAL
    TYPES --> RESP
    ZOD --> FORM
    TYPES --> HOOKS

    style PKG fill:#f59e0b,stroke:#000,color:#000
    style API_USE fill:#10b981,stroke:#000,color:#fff
    style WEB_USE fill:#6366f1,stroke:#000,color:#fff

```

Schemas define runtime validation rules. **Types** are inferred from schemas using `z.infer<>`, ensuring they can never drift.

Enums, constraints (min/max length, email format, positive numbers), and optional fields are all defined once and consumed by both the API validation middleware and the frontend form resolvers.

Schema Coverage

```

graph TD
    subgraph Schemas["8 Schema Files"]
        US["user.schema.ts  
register · login · updateProfile"]
        AS["account.schema.ts  
create · update · types enum"]
        TS["transaction.schema.ts  
create · update · query · sort"]
        CS["category.schema.ts  
create · update · usage metadata"]
        BS["budget.schema.ts  
create · update · period enum"]
    end

```

```

    GS["goal.schema.ts
create · update · addFunds"]
    RS["recurring.schema.ts
create · update · frequency enum"]
    ADS["advisor.schema.ts
chat, action proposals, execution"]
end

subgraph Wrappers["Generic API Types"]
    AR["ApiResponse<T>"]
    PR["PaginatedResponse<T>"]
    AE["ApiError"]
end
```

Testing Strategy

```

pie title Test Distribution (442 total)
  "Shared Types - Schema Validation" : 162
  "API - Services & Business Logic" : 106
  "MCP - Tools & Resources" : 61
  "API - Middleware & Utilities" : 41
  "Web - Utility Functions" : 41
  "Agentic AI - Agents & Pipeline" : 31
```

Layer	Tool	What's Tested
API services	Vitest + mongodb-memory-server	Business logic with real Mongoose operations
API middleware	Vitest	Auth token validation, Zod middleware, error shape
API utilities	Vitest	ApiError factories, pagination helpers
Shared schemas	Vitest	Every Zod schema: valid, invalid, edge cases
Web utilities	Vitest + jsdom	Formatting, class merging, helpers
MCP tools	Vitest + mongodb-memory-server	Tool handlers, resource providers, auth resolver
Agentic AI	Vitest	Agent orchestration, classification, MCP client

Test Execution Flow

```

graph LR
  CMD["npm run test"] --> TURBO["Turborepo runs in parallel"]
```

```
TURBO --> ST_TEST["shared-types
vitest (node)
162 tests"]
  TURBO --> API_TEST["api
vitest (node)
147 tests"]
    TURBO --> WEB_TEST["web
vitest (jsdom)
41 tests"]
      TURBO --> MCP_TEST["mcp
vitest (node)
61 tests"]
        TURBO --> AI_TEST["agentic-ai
vitest (node)
31 tests"]

  API_TEST --> MMS["MongoMemoryServer
in-memory MongoDB"]
    MCP_TEST --> MMS
      MMS --> CLEAN["afterEach:
clear collections"]

style CMD fill:#6366f1,stroke:#000,color:#fff
style ST_TEST fill:#f59e0b,stroke:#000,color:#000
style API_TEST fill:#10b981,stroke:#000,color:#fff
style WEB_TEST fill:#8b5cf6,stroke:#000,color:#fff
style MCP_TEST fill:#4f46e5,stroke:#000,color:#fff
style AI_TEST fill:#cc785c,stroke:#000,color:#fff
```

API tests run against a real MongoDB instance (in-memory) with full Mongoose operations - no mocking the database layer. Each test file gets a clean database via **afterEach** collection clearing.